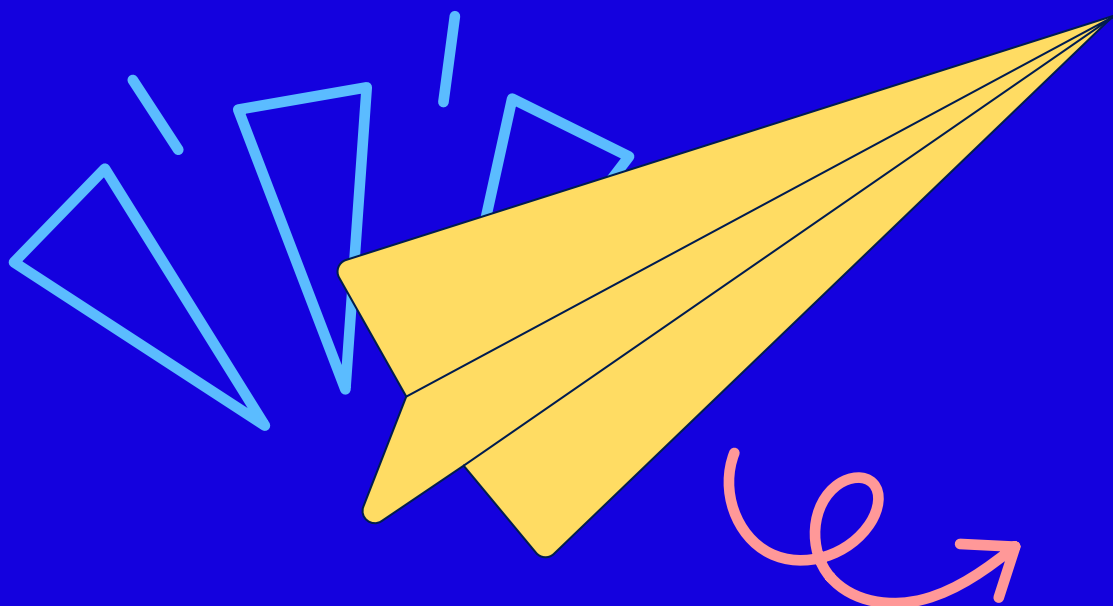




Maquetación: cómo hacer inclusivos nuestros sitios web

Frontend



Maquetación: cómo hacer inclusivos nuestros sitios web

En este capítulo se da por sentado que se tienen los conocimientos, no voy a decir básicos porque no lo son, sino fundamentales de la maquetación, es decir, HTML y CSS. Aquí trataremos formas de poder optimizar y profesionalizar aún más nuestro trabajo, enfocándonos en adaptar y hacer inclusivos nuestros sitios web.

Autor: Miguel Ángel Muñoz



Preprocesadores CSS



Responsividad



Accesibilidad



Frameworks



¿Qué hemos aprendido?

Preprocesadores CSS

Para estudiar este fastbook, te recomendamos que accedas a las siguientes plataformas para obtener las **herramientas necesarias**:



Editor de código fuente desarrollado por Microsoft para Windows, Linux, macOS y
Web: [Visual Studio Code](#)



Herramientas para realizar pruebas de accesibilidad digital: [Axe DevTools](#)

Un preprocesador de CSS permite generar hojas de estilo en cascada a partir de una sintaxis que proporciona el propio preprocesador, la cual sirve de potenciador simplificando el estilado y extendiendo las posibilidades de todo lo que se puede hacer, además de añadir legibilidad al código.

Algunos de los **preprocesadores** más conocidos son los siguientes:

- [SASS](#)
- [LESS](#)
- [Stylus](#)
- [PostCSS](#)

Estos preprocesadores hay que **instalarlos y ejecutarlos** para que transformen lo que hayamos escrito en código CSS que sea legible por el navegador. Para ello, lo más común y sencillo es instalar el preprocesador que vayamos a utilizar a través de **NodeJS** y después ejecutar un comando o, si lo queremos hacer más operativo, añadir un nuevo script o paso en un script ya existente (como el *build*) para que ejecute el preprocesador y genere los archivos CSS.

Para no complicarnos en entender múltiples preprocesadores al mismo tiempo, vamos a centrarnos en uno, en SASS. Después, pasarnos a otro preprocesador será muy sencillo ya que aporta funcionalidades muy similares, lo único que cambia es la sintaxis de utilización.

SASS

Lo primero que se suele destacar en los preprocesadores es el uso de variables, aunque podemos perfectamente crear también *custom* variables en CSS utilizando el **guion doble** para definir las (`--mivariable: algo`) y la **función var** para utilizarlas (`var(--mivariable)`); no obstante se hace más sencillo con los preprocesadores.

En SASS tenemos la posibilidad de utilizar dos sintaxis diferentes y podemos elegir la que más nos gusta, únicamente tendremos que marcar la extensión como corresponda. La diferencia principal es si usamos **brackets {}** o **indentación**.



Uso de brackets y extensión .scss:

```
.my-variable {  
    width: 100%;  
}
```



Uso de indentación y extensión .sass:

```
.my-variable  
  width: 100%
```

La segunda opción es más limpia, aunque quizás menos clara ya que no están los brackets para delimitar el principio y el final del bloque. Cuestión de gustos.

Instalación

Para instalar SASS en tu ordenador y que puedas hacer pruebas, utilizaremos el gestor de **paquetes NPM** instalando SASS de forma global para tenerlo disponible en todos los proyectos. También puedes instalarlo de manera local en el proyecto y utilizarlo a través de un script incluido en el package.json.

El comando de instalación supongo que ya te lo imaginarás:

```
npm i -g sass
```

Con esto lo tendrás disponible a nivel global en cualquier proyecto, pudiendo ejecutar el siguiente comando para ejecutarlo y transformar archivos SASS en CSS:

```
sass input.scss output.css
```

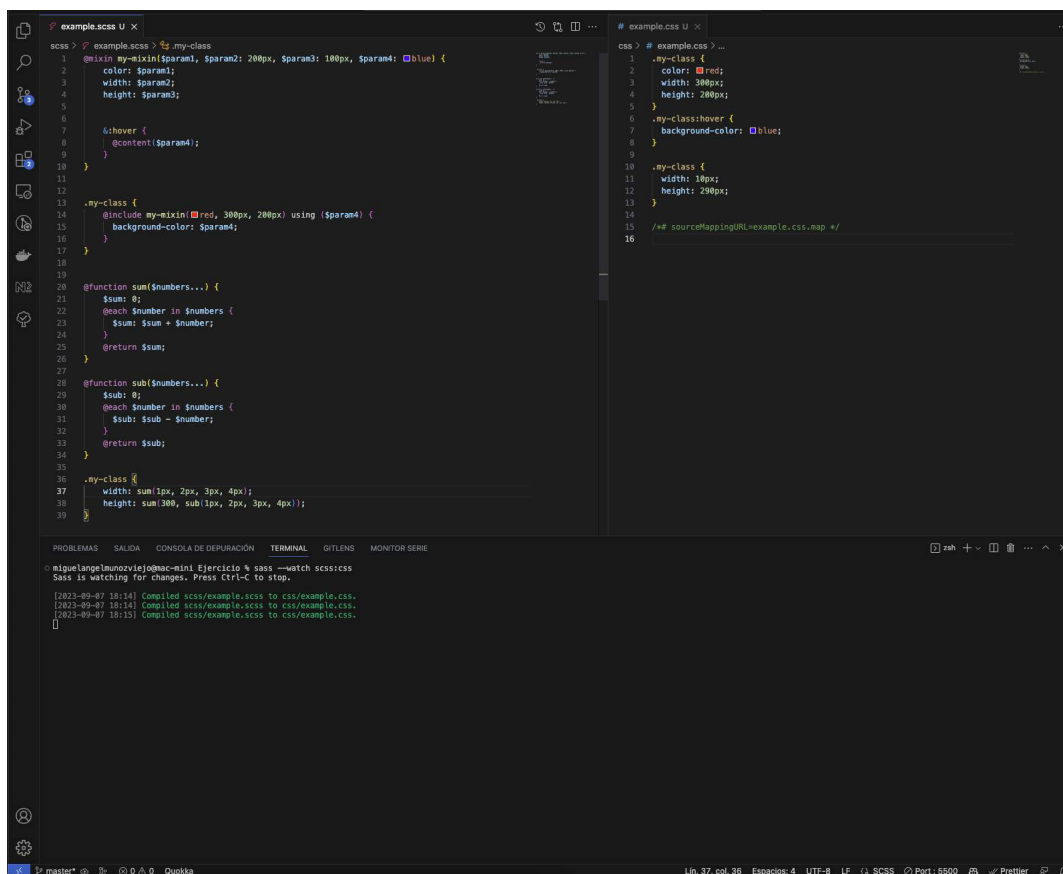
Con este comando se compilará el archivo `input.scss` y el resultado se añadirá en el **archivo `output.css`**

Lo más práctico es usar el **modificador `--watch`** para que esté observando cada cambio que hagamos y compilar automáticamente. Lo más habitual es que tengamos todos nuestros archivos SASS en una carpeta, por ejemplo, en **una carpeta con el nombre `scss`**.

Aquí podemos hacer que detecte cualquier cambio en una carpeta concreta y compile en otra carpeta, por ejemplo:

```
sass --watch scss:css
```

Este comando lo que hará es observar cualquier cambio en archivos SASS en la carpeta `scss` y, cuando este se produzca, ejecutar la compilación y guardarlo con el mismo nombre, pero con extensión `.css` en la carpeta `css`.



Variables

Las variables son algo diferentes a las variables custom de CSS, aparte de cómo se definen (*\$mi-variable: 100px*) y de cómo se usan (*\$mi-variable* directamente sin tener que usar ninguna función):

- Se compilan antes de que lleguen al CSS, es decir, si en el SASS usamos la variable *\$mi-variable: 100px*, en el CSS ya compilado veremos el valor que tenía esa variable en SASS, es decir, en este caso, 100px.
- Las variables CSS se tienen que declarar bajo un elemento y, por ello, pueden tener diferente valor en función del elemento en el que se encuentren. En SASS pueden ser definidas a nivel global. Además, mantienen el valor que tengan ese momento, es decir: si defines una variable, la utilizas y, en unas líneas más abajo, le asignas un nuevo valor y la vuelves a utilizar, la variable tendrá diferente valor en cada caso.
- Las variables en los preprocesadores pueden obtener más valores como booleanos, por ejemplo, a diferencia de las variables de CSS. Esto permite poder utilizarlas en funciones y condicionales.
- También se pueden construir nombres de variables de forma dinámica con la combinación de otras variables utilizando **#{}** , por ejemplo: **#{\$prefix}-#{\$propname}: \$value**.



Hay unas cuantas características más que son especiales en SASS que te recomiendo que las leas más en profundidad en su [documentación](#).

Nesting

Si has estado practicando con CSS, imagino que en algún momento te habrás encontrado con ciertas cosas que te habrán hecho pensar que podrían estar más optimizadas, ya que en ocasiones te toca repetir código.

Un ejemplo sería cuando tienes que repetir una y otra vez el mismo selector padre cuando quieres dar estilos a varios elementos hijos, por ejemplo:

```
#main-menu {  
  position: sticky;  
  top: 0;  
  background-color: color-mix(in srgb, var(--extra-dark-color) 95%, transparent);  
}  
  
#main-menu h2 {  
  color: var(--light-color);  
}
```

En este caso, se tiene que repetir el selector *#main-menu* múltiples veces, y es un verdadero rollo. Esto es el *nesting*, poder anidar elementos, por ejemplo, tomando de base el ejemplo anterior:

```
#main-menu {  
  position: sticky;  
  top: 0;  
  background-color: color-mix(in srgb, var(--extra-dark-color) 95%, transparent);  
  
  h2 {  
    color: var(--light-color);  
  }  
}
```

El h2 queda anidado debajo de *#main-menu* y sería lo mismo que en el caso anterior pero...
¡Más óptimo!

Si quisiéramos replicar algo como esto:

```
#main-menu > div:last-child {  
float: right;  
text-align: right;  
}
```

Con un preprocesador, usando *nesting*, podríamos expresarlo de la siguiente manera:

```
#main-menu {  
  & > div {  
    &:last-child {  
      float: right;  
      text-align: right;  
    }  
  }  
}
```

Si queremos representar de forma anidada un selector que use operadores de relación (>, +...) o modificadores (:hover, :after...), tenemos que utilizar la referencia al **elemento padre**, **que se representa con &**.

Placeholder Selectors

¿Alguna vez te has encontrado con la situación de que dos elementos tienen las mismas propiedades a excepción de unas pocas? Esto con CSS lo resolvemos de esta forma:

```
.element-a, .element-b {  
  width: 100%;  
  height: 200px;  
  background-color: var(--gold-color);  
}  
  
.element-a:hover, .element-b:hover {  
  background-color: var(--dark-color);  
  transition: all 0.2s linear 0s;  
}  
  
.element-a {  
  color: var(--dark-color);  
}  
  
.element-a:hover {  
  color: var(--gold-color);  
}  
  
.element-b {  
  color: var(--white-color);  
}  
  
.element-b:hover {  
  color: var(--gold-color);  
}
```

En SASS podemos escribirlo de una forma mucho más intuitiva, legible y optimizada:

```
%card {
  width: 100%;
  height: 200px;
  background-color: var(--gold-color);

  &:hover {
    background-color: var(--dark-color);
    transition: all 0.2s linear 0s;
  }
}

.element-a {
  @extend %card;
  color: var(--dark-color);
}

.element-b {
  @extend %card;
  color: var(--white-color);
}
```

Módulos

SASS está construido basado en módulos, que ya os debería sonar de JavaScript ESM. Por ello podemos organizar los archivos SASS de una forma mucho más modular e intuitiva.

Tenemos a nuestra disposición una serie de reglas que nos permiten utilizar los módulos:



@use 'url': nos permite cargar y utilizar otros archivos SASS.



@forward "url": permite exponer *mixins*, *functions* y variables de otros archivos SASS para que puedan ser utilizados cuando se utilice *@use*.



@import 'url': es una extensión del *@import* del que disponemos en CSS que permite importar archivos SASS a parte de CSS. Además los combina en un mismo archivo en lugar de que lo tenga que hacer el navegador.

Mixins

Los *mixins* están pensados para definir partes de código para que sean reutilizables, incluso permiten recibir parámetros (obligatorios y opcionales definiendo un valor por defecto) para poder utilizarlos en la definición de propiedades del propio mixin.

También, al igual que en los web *components*, podemos definir un *slot* en el que poder introducir contenido *custom* a través de la regla *@content*.

```
@mixin my-mixin($param1, $param2: 200px, $param3: 100px, $param4: blue) {  
  color: $param1;  
  width: $param2;  
  height: $param3;  
  &:hover {  
    @content($param4);  
  }  
}  
  
.my-class {  
  @include my-mixin(red, 300px, 200px) using ($param4) {  
    background-color: $param4;  
  }  
}
```

Esto se traduce en...

```
.my-class {  
  color: red;  
  width: 300px;  
  height: 200px;  
}  
  
.my-class:hover {  
  background-color: blue;  
}
```

Como ves, nos abre un abanico de posibilidades enorme para poder hacer complejos y/o tediosos estilos de una forma mucho más rápida y, en cuanto lo tengamos asimilado, también sencilla.

La estructura es simple: con la regla `@mixin` indicamos que vamos a iniciar un mixin y le damos un nombre y, si lo necesitamos, una serie de parámetros. Dentro del mixin definimos el código SASS que necesitamos.

Después, para usarlo, únicamente tenemos que aplicar la regla **@include** indicando el nombre del mixin y pasando los parámetros que requiere. Lo podemos utilizar tantas veces como queramos y así evitamos mucha duplicidad de código.

Funciones

Las funciones son similares a los *mixins*, pero con la diferencia de que están pensadas para retornar un valor y ejecutar procesamientos más complejos que poco tienen que ver con estilos. Además, utilizan la regla `@function` y para aplicarla no requerimos de la regla `@include`, únicamente tenemos que invocarlas usando su nombre.

```
@function sum($numbers...) {  
  $sum: 0;  
  @each $number in $numbers {  
    $sum: $sum + $number;  
  }  
  @return $sum;  
}  
  
.my-class {  
  width: sum(1px, 2px, 3px, 4px);  
}
```

El código anterior se traduce en...

```
.my-class {  
  width: 10px;  
}
```

Como ves, aquí nos estamos aproximando mucho a la programación y, por tanto, empezamos a utilizar reglas que manejan el flujo de procesamiento, por ejemplo:

- **@if y @else:** así podemos usar condicionales utilizando variables.
- **@each:** para recorrer un listado de elementos.
- **@for:** esta regla nos permite replicar un bloque *for* para repetir la ejecución de un bloque *n* veces.
- **@while:** lo mismo que en programación en otros lenguajes, esta regla repite la ejecución de un bloque mientras se cumpla la condición indicada.

Hasta ahora, hemos visto una descripción de las funcionalidades principales que nos aportan los preprocesadores de estilos con las que puedes empezar a entrenar y crear archivos SASS tan complejos que hagan verdaderas virguerías.

Hasta ahora, hemos visto una descripción de las funcionalidades principales que nos aportan los preprocesadores de estilos con las que puedes empezar a entrenar y crear archivos SASS tan complejos que hagan verdaderas virguerías.

Pero, sobre todo, practica, practica y vuelve a practicar hasta que entiendas cómo funciona. Mi recomendación es que te plantees diferentes ejemplos de CSS y cómo se pueden optimizar con SASS para mejorar tus habilidades.

i Continúa repasando la [documentación](#) de SASS, ya que hay más reglas y funcionalidades disponibles que te serán de gran utilidad.

Responsividad

El diseño web responsivo es la clave, el santo grial del maquetador, es un imperativo que debemos tener en cuenta en todas las fases de desarrollo web, desde el diseño hasta la construcción.

En la actualidad disponemos de una gran cantidad de formatos de dispositivos: ordenadores fijos, portátiles, móviles, tabletas, monitores y todos y cada uno de ellos con un abanico inmenso de resoluciones posibles. Por tanto, es impensable diseñar y construir un sitio web teniendo en cuenta un único tamaño de pantalla, tenemos que perseguir dos cosas: **flexibilidad y adaptabilidad**.

1

Flexibilidad

La construcción de los sitios web debe ser flexible, es decir, tenemos que huir, y cuanto más lejos mejor, de la utilización de medidas fijas y perseguir la construcción de elementos versátiles, que se adapten en función del tamaño de la pantalla. Un contenedor debe poder extenderse y adaptarse reorganizando sus elementos en función del tamaño del que disponga.

No podemos conseguir un buen diseño y construcción de un sitio web si únicamente garantizamos la flexibilidad. Por lo tanto, es necesario que se produzcan cambios en función de los rangos de los tamaños de pantalla, es decir, no podemos pensar de igual manera en cómo vamos a estructurar el contenido en un monitor que en un dispositivo móvil, hay diseños que dejan de tener sentido en resoluciones bajas o altas.

Breakpoints

La parte de flexibilidad se trabaja con las técnicas de *layout* (float, flexbox, CSS Grid), pero ¿y la adaptabilidad?

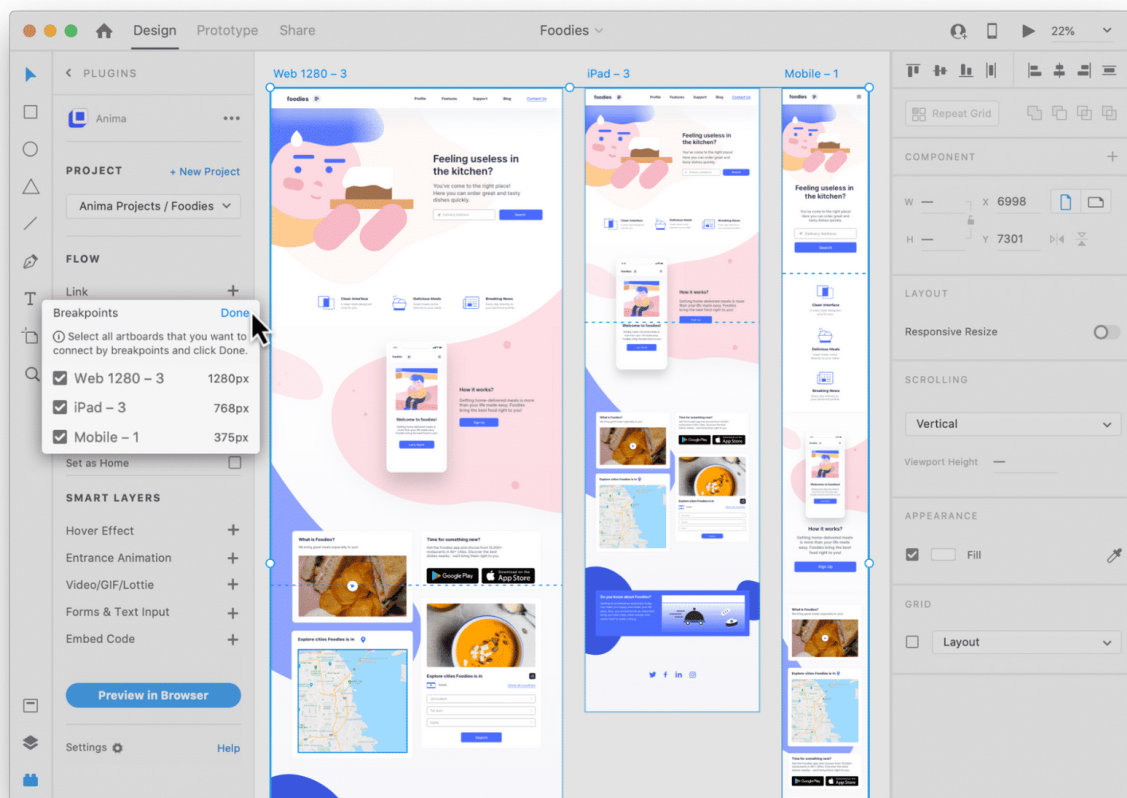
Para hacer nuestro sitio adaptable tenemos que definir lo que se llaman *breakpoints* o puntos de corte que tienen en cuenta rangos de tamaños de pantalla. Estos nos permiten adaptar el diseño y la construcción en función de estos rangos por lo que debe de existir una especie de contrato entre el equipo de diseño y el equipo de desarrollo para definir estos puntos de corte. Un ejemplo:

- **sm - 576px:** principalmente móviles.
- **md - 768px:** principalmente tabletas.
- **lg - 993px:** enfocado a monitores pequeños.
- **xl - 1200px:** enfocado a monitores estándar.
- **xxl - 1400px:** enfocado a monitores grandes.

Por poner un ejemplo, tendríamos un diseño que contemplase los dispositivos móviles que, siguiendo estos puntos de corte, incluiría desde 0 hasta 576px. Después, contemplaríamos las tabletas dentro del rango entre 576px y 768px y así sucesivamente.

Por ejemplo, en la siguiente imagen se puede ver un planteamiento en el que se tienen en cuenta 3 rangos de tamaño:

- Móvil
- iPad
- Web



Fuente: animaapp.com

Media queries

Vale, ya nos ha quedado claro qué son el diseño responsivo, la flexibilidad, la adaptabilidad y los *breakpoints*, pero... ¿Cómo se aplica todo esto en la maquetación web?

Aquí entran en juego las [media queries](#) o las **consultas a los medios**, es decir, consultas al tamaño del visor. Nos permiten definir reglas de tamaño y características de medias para definir estilos en función de ellas.

Antes de entrar en esto, tengamos en cuenta que los medios no son solo pantallas de un dispositivo, también puede ser un documento impreso, por ejemplo, para cuando queremos generar un PDF.

Veamos con detenimiento este ejemplo de *media query*:

```
@media screen and (min-width: 576px) {  
  .container {  
    max-width: var(--breakpoint-sm);  
  }  
  
  .container-sm {  
    max-width: var(--breakpoint-sm);  
  }  
}
```

¿Qué se está definiendo? Si te fijas, para definir un media tenemos que iniciar con la regla *@media* (esto nos recuerda a los preprocesadores de CSS pero no, es CSS puro), además de que estamos especificando *screen* (una pantalla) y que el ancho de la pantalla tiene como mínimo 576px, es decir: todas las propiedades que indicamos dentro de este media se aplicarán únicamente cuando se cumplan estas condiciones.

Con esto ya tenemos todas las armas necesarias para construir sitios web completamente *responsive*.



Revisa la documentación sobre *media queries* [aquí](#).

Utiliza las herramientas de desarrollador para hacer las pruebas con diferentes resoluciones.

Mobile first

Ya tienes las herramientas, te has estudiado a la perfección las técnicas de layout, los breakpoints, las media queries... Se podría decir que ya estás preparado para construir sitios web completamente responsivos (sino para aquí y vuelve a repasar lo que necesites).

☐

¿Te ha surgido alguna duda a la hora de empezar a plantear los estilos?

☐

¿Por dónde empiezas a definir los estilos, qué es lo primero que tienes en cuenta, qué tamaño?

☐

No puedes estilar todo a la vez, tienes que elegir un tamaño que sea la referencia para el resto. ¿Cuál crees que debería ser?

Teniendo en cuenta que nos pasamos más de media vida y que tenemos toda nuestra información en los móviles, la metodología *mobile first* propone que la pantalla más importante es la del móvil y, por eso, se tiene que empezar a diseñar y a construir teniendo como referencia esta pantalla. Por tanto:

1

Primero creas todos los estilos (aún sin tener en cuenta ni *breakpoints* ni *media queries*) teniendo como referencia (y ajustando la pantalla a través de las herramientas del desarrollador) la pantalla de un móvil.

2

Una vez tienes perfectamente diseñado y construido este tamaño, entonces es cuando empiezas a definir *breakpoints* y *media queries* para el resto.

Accesibilidad

Cambiando de tercio completamente, no podemos acabar la parte de maquetación sin dedicar una sección a la accesibilidad, ya que es fundamental y, de hecho, debería ser un *must* en todos nuestros proyectos.

Tenemos que **perseguir la inclusión de todos** los miembros de nuestra sociedad y está en nuestras manos aportar nuestro granito de arena facilitando la usabilidad a esas personas que tienen ciertas discapacidades: ciegos, daltónicos...

Las personas ciegas, por ejemplo, tienen que poder navegar perfectamente por nuestros sitios web. Utilizan herramientas especiales que les van narrando lo que contiene la página y, así, pueden imaginarse y entender el contenido de esta. De ahí que debemos garantizar que estas herramientas pueden leer nuestro sitio web perfectamente y que debemos asegurar esas medidas de accesibilidad.

Ya que a nadie nos gusta quedarnos atrapados en una página en la que no disponemos de ninguna forma de avanzar o retroceder, trabajando la accesibilidad del sitio evitamos este tipo de situaciones. Por lo tanto, te recomiendo encarecidamente que leas la documentación de Mozilla sobre la accesibilidad , ya que trata este tema en profundidad, tanto para que entendamos qué es la accesibilidad como para su aplicación en el desarrollo web.

 **Consulta aquí la [documentación](#) de Mozilla sobre la accesibilidad.**

Como decíamos, afortunadamente, tenemos a nuestra disposición herramientas que nos facilitan garantizar que nuestro sitio web es accesible. Aquí te presentamos dos de ellas:

Lighthouse

[Lighthouse](#) está disponible en las herramientas del desarrollador y, además de medir el SEO y el performance, también comprueba si tenemos incidencias de accesibilidad.

Axe

[Axe](#) es una herramienta del desarrollador que se centra específicamente en la accesibilidad, es muy potente, de hecho, nos permite incluirla en los test unitarios y en los test e2e para poder hacer comprobaciones automáticas antes de publicar nuestro sitio web.

Axe, además, nos ofrece la posibilidad de establecer el nivel de reglas que queremos aplicar.

En accesibilidad, existen 3 niveles regulados por la [WCAG](#), que pueden llegar a ser exigidos por las empresas, sobre todo si van a ser para algún cliente público. Por lo general se exige un nivel mínimo de AA o doble A. Los niveles son tres:

- **Nivel 1: A**
- **Nivel 2: AA**
- **Nivel 3: AAA**

Para cumplir con las [reglas de accesibilidad](#) debemos tener en cuenta cada una de ellas (las puedes consultar en el listado que contempla Axe o en la propia web de la WCAG). Aquí te destacamos algunas de ellas, ¡apunta!

☐

Vigilar el contraste de colores: existen herramientas de medición, pero la propia herramienta de Axe o Lighthouse analiza estos contrastes.

☐

Las imágenes deben tener siempre un **alt** que describa perfectamente lo que se muestra (es lo que leen las herramientas de apoyo).

☐

Añadir **captions** para tener referencias del contenido.

☐

Hacer una estructuración **semántica correcta**.

☐

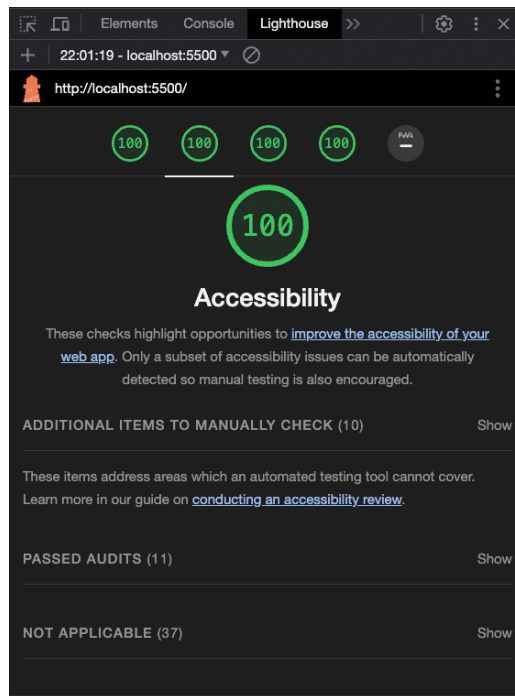
Establecer el **atributo lang de la tag html** con el lenguaje correcto.

☐

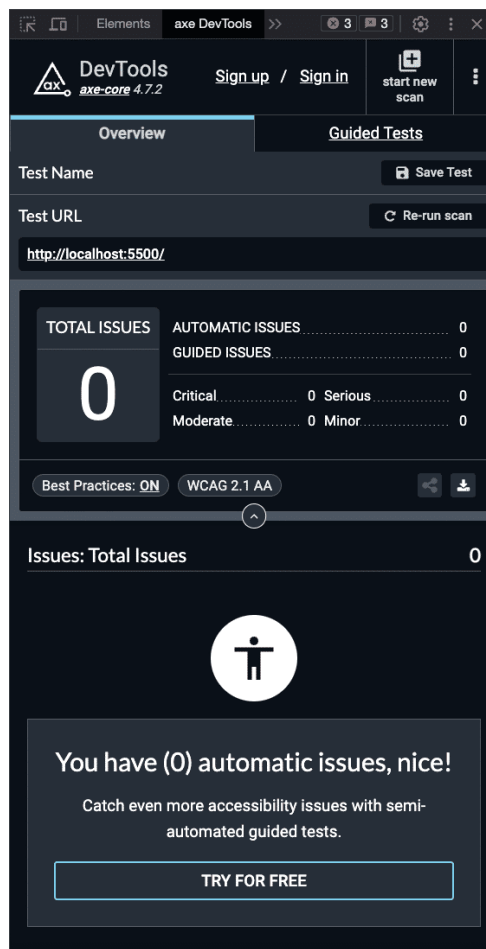
Los **anchor** deben ser únicamente usados como enlaces y diferenciarse perfectamente del resto del contenido.

☐

Las **labels de los formularios** deben ser correctas y no estar duplicadas.



Lighthouse



Frameworks

No podemos cerrar este tema dedicado a la maquetación sin hablar de los frameworks. Aunque es fundamental que el aprendizaje lo iniciemos sin ningún tipo de framework, que desarrollemos y probemos todas las opciones que tenemos a nuestra disposición de la forma más pura posible, es más que probable que en ciertas ocasiones nos pueda ser de utilidad el uso de algún framework.

¿Y qué es un framework?

Un framework es como si fuera una caja de herramientas que pone a nuestra disposición funcionalidades ya hechas, sin necesidad de que las creamos por nosotros mismos.

Eso sí, estamos obligados a utilizar la estructura y planteamiento que han definido, un aspecto que marca una diferencia fundamental con una librería.

Para proyectos importantes, se construye un framework adaptado al cliente, no se utiliza ninguno de los que vamos a mencionar. Es por esto por lo que quiero recalcar la importancia de que procuremos olvidarnos de los frameworks, salvo en proyectos personales o que no tengan la entidad suficiente como para requerir un **framework propio**. De hecho, una práctica muy interesante, si de verdad quieres llegar a ser un gran maquetador, podría ser crear tu propio framework.

Para que tengas algunas referencias en las que fijarte o que puedas utilizar, te recomiendo algunos de los más utilizados:

Bootstrap —

[Bootstrap](#) es el más extendido y conocido de los frameworks. Te permite personalizarlo y pone a tu disposición una enorme cantidad de recursos.

Tailwind —

[Tailwind](#), a pesar de ser más reciente, ha ganado mucha fuerza en el mercado, quizá porque se centra en ofrecer mucha operatividad para poder aplicar estilos desde el HTML, es decir, expone múltiples clases con estilos preconfigurados. También dispone de una propuesta de componentes que podemos utilizar directamente.

¿Qué hemos aprendido?

A lo largo de este fastbook hemos aprendido lo siguiente:



Hemos estado viendo cómo podemos optimizar y hacer más mantenible nuestro trabajo como maquettadores con el **uso de los preprocesadores de CSS**. Estos amigos nos abren las puertas para crear unos estilos que utilizan la lógica y, por tanto, podemos complejizar todo lo que se nos ocurra para ser capaces de crear proyectos muy complejos y de una forma mucho más rápida.



También, hemos visto la clave de la maquetación, la columna vertebral: hemos aprendido cómo conseguir que nuestros **sitios web sean flexibles y se adapten** a todos los entornos en los que se pueden llegar a encontrar.



Y, por último, no podíamos olvidarnos de un tema sensible para la sociedad actual: **la accesibilidad**. Los maquettadores profesionales siempre, siempre, debemos pensar en todos los usuarios que pueden ver nuestro trabajo, debemos esforzarnos en construir un mundo más fácil para aquellos que no lo tienen tanto.

Sigue aprendiendo y profundizando en tus conocimientos, te animo a explorar en la documentación y recursos disponibles.

¡Enhorabuena! Fastbook superado